

8장_성능 최적화

8-1. 성능 최적화

▼ 8.1.1. 데이터를 활용한 성능 최적화

- 최대한 많은 데이터 수집
- 데이터 생성: 5장의 이미지 조작 코드 참고
- 데이터 범위(scale) 조정: by 활성화 함수
 - 시그모이드 사용: 데이터셋 범위를 0~1로 조정
 - 하이퍼볼릭 탄젠트 사용: 데이터셋 범위를 -1~1로 조정
- 외에도 정규화, 규제화, 표준화도 도움

▼ 8.1.2. 알고리즘을 이용한 성능 최적화

- 유사한 용도의 알고리즘을 선택하여 훈련시켜보고, 최적 성능 알고리즘 선택

▼ 8.1.3. 알고리즘 튜닝을 위한 성능 최적화



성능 최적화에 가장 많은 시간이 소요되는 부분!

아래는 선택할 수 있는 하이퍼파라미터이다.

- 진단
: 성능 향상이 어느 순간 멈춘 경우, 원인 분석
 - 훈련성능 >>>>> 검증성능: 과적합 의심 → 규제화 진행
 - 훈련, 검증성능 모두 저조: 과소적합 의심 → 구조 변경, 에포크 조정
 - 훈련성능 > 검증성능인 변곡점이 있는 경우: 조기종료 고려
- 가중치

- 가중치에 대한 초깃값은 작은 난수 사용
- 오토인코더 같은 비지도 학습을 이용해 사전훈련 후 지도학습도 가능
- 학습률
 - : 네트워크 구성에 따라 다르므로, 학습 결과를 보고 조금씩 변경
 - 네트워크 계층이 많다면 학습률은 높아야 함
 - 네트워크 계층이 몇 개 되지 않는다면 학습률은 작게 설정
- 활성화 함수
 - : 신중하게 변경해야 함. 손실 함수도 함께 변경해야 하는 경우가 많기 때문.
 - 일반적으로 시그모이드나 하이퍼볼릭 탄젠트를 사용했다면,
 - 출력층에서는 소프트맥스나 시그모이드를 많이 선택
- 배치, 에포크
 - 최근 딥러닝 트렌드: 큰 에포크, 작은 배치
 - 다양한 테스트를 진행해 보는 게 좋음
- 옵티마이저, 손실 함수
 - 일반적 옵티마이저: SGD
 - Adam, RMSProp 등도 좋은 성능
 - 다양하게 적용해보고 성능 최고인 것을 선택해야 함
- 네트워크 구성
 - : 네트워크 토폴로지(topology) 라고도 함.
 - 변경해 가면서 성능 테스트
 - ex) 하나의 은닉층에 여러 뉴런을 포함시키거나(넓은 네트워크)
 - ex) 계층을 늘리되 뉴런 개수를 줄여보기(깊은 네트워크)
 - ex) 둘을 결합해도 됨

▼ 8.1.4. 앙상블을 이용한 성능 최적화



성능 향상은 단시간에 해결 X

8-2. 하드웨어를 이용한 성능 최적화



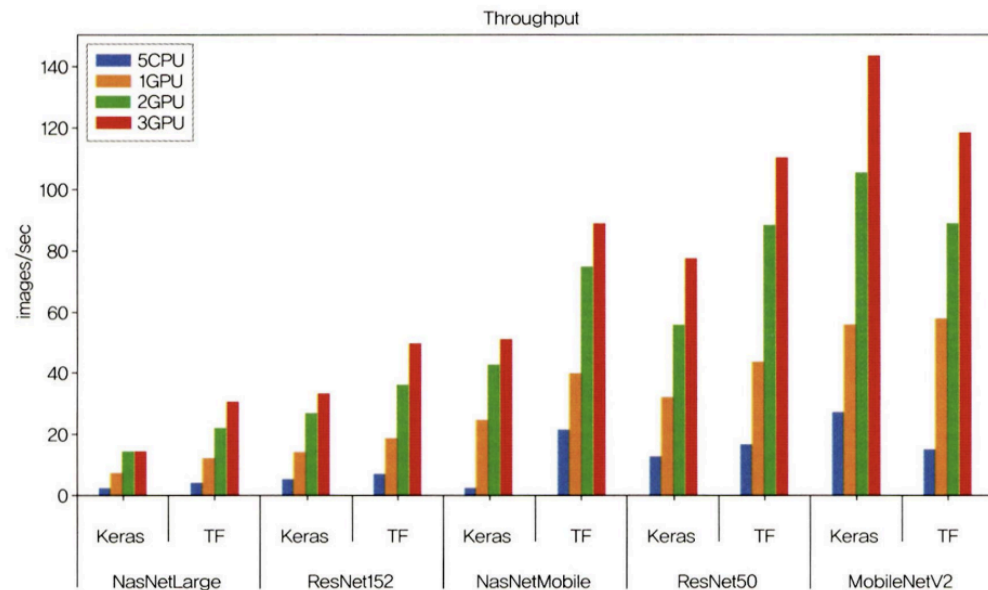
CPU가 아닌 GPU를 이용

▼ 8.2.1. CPU와 GPU 사용의 차이

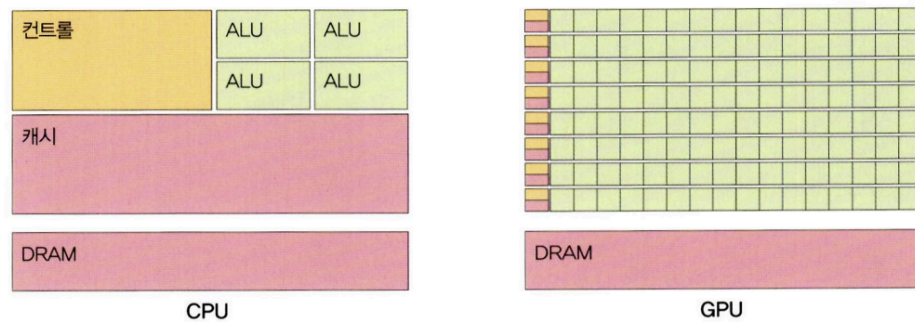


CPU와 GPU는 개발 목적, 내부 구조가 다르다.

▼ 그림 8-3 CPU와 GPU 성능 비교¹⁾



▼ 그림 8-4 CPU와 GPU의 구조 비교



- CPU
 - CPU의 구성
 - ALU: 연산 담당
 - control: 명령어 해석, 실행
 - 캐시: 데이터를 담아 둠

- 직렬 처리 방식: 명령어 입력 순서대로 데이터 처리
 - 한번에 하나의 명령어만 처리
 - → 연산을 담당하는 ALU가 많을 필요 X
 - GPU
 - GPU의 구성(차이점 중심)
 - ALU 개수가 많아짐
 - 캐시 메모리 비중 낮음
 - 병렬 처리 방식: 여러 명령을 동시에 처리
 - 많은 ALU로 구성 → 병렬 처리에 특화
 - 하나의 코어에 ALU 수백~수천개 → 3D 그래픽 작업 등 빠름
 - 비교
 - 개별적 코어 속도: CPU > GPU
 - 직렬 연산을 수행하는 경우(CPU가 적합한 경우): 행렬 연산 → 파이썬, 매트랩
 - 병렬 연산을 수행하는 경우: 역전파처럼 복잡한 미적분
- ▼ 8.2.2. GPU를 이용한 성능 최적화
- 설치이므로 생략한다.

8-3. 하이퍼파라미터를 이용한 성능 최적화

▼ 8.3.1. 배치 정규화를 이용한 성능 최적화



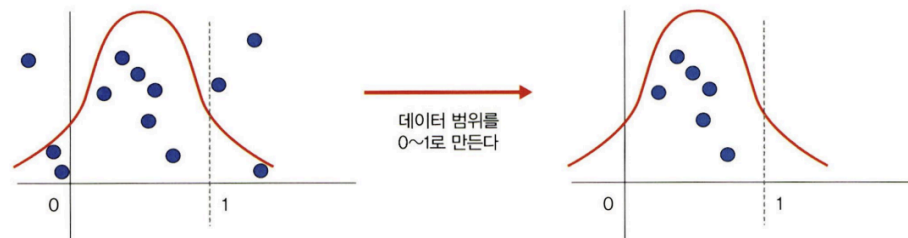
배치 정규화에 앞서, 유사한 의미로 사용되는 용어를 알아보자.

- 정규화

: 데이터 범위를 사용자가 원하는 범위로 제한하는 것.

- ex) 이미지 데이터는 픽셀 정보로 0~255 사이의 값을 가짐
→ 255로 나누어 0~1.0 사이의 값으로 만듦
- 특성 스케일링이라고도 함
- 스케일 조정을 위해 MinMaxScaler() 기법 사용

▼ 그림 8-31 정규화

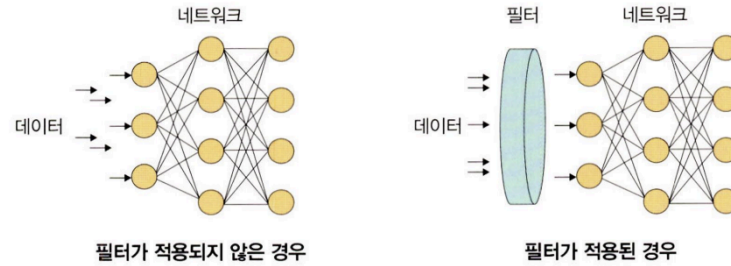


- 규제화

: 모델 복잡도를 줄이기 위해 제약을 두는 방법

- 제약: 데이터가 네트워크에 들어가기 전 필터를 적용한 것

♥ 그림 8-32 규제화



규제를 이용하여 모델 복잡도를 줄이는 방법은 다음과 같습니다.

- 드롭아웃
- 조기 종료

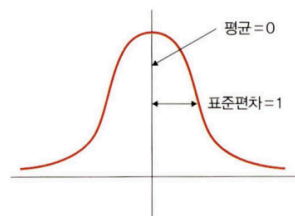
※ 드롭아웃은 8.3.2절에서, 조기 종료는 8.3.3절에서 자세히 다룹니다.

• 표준화

: 기존 데이터를 평균 0, 표준편차 1의 형태로 만드는 것

- 표준화 스칼라, z-score 정규화라고도 함

♥ 그림 8-33 표준화



평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 사용합니다. 보통 데이터 분포가 가우시안 분포를 따를 때 유용한 방법으로 다음 수식을 사용합니다.

$$\frac{x-m}{\sigma}$$

(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

- 배치 정규화: 데이터 분포가 안정되어 학습 속도 향상 가능

- 기울기 소멸or폭발 해결 위한 방법
- 일반적으로 이 문제 해결 위해 렐루 손실함수 사용, 초깃값 튜닝, 학습률 조정함
- 기울기 소멸, 폭발 원인: 내부 공변량 변화
 - 네트워크의 각 층마다 활성화 함수가 적용되며 입력 값의 분포가 계속 바뀜
 - ⇒ 따라서, 분산된 분포를 정규분포로 만들자.
- ⇒ 표준화와 유사한 방식을 미니 배치에 적용
 - 평균 0, 표준편차 1로 유지
 - 수식

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{----①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{----②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{----③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{----④}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

- ① 미니 배치 평균을 구합니다.
- ② 미니 배치의 분산과 표준편차를 구합니다.
- ③ 정규화를 수행합니다.
- ④ 스케일(scale)을 조정(데이터 분포 조정)합니다.

• 장단점

- 장점) 매 단계마다 활성화 함수를 거치며 데이터셋 분포 일정해짐 → 속도 향상
- 단점1) 배치 크기가 작을 때 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있음
 - ex) 분산이 0이면 정규화 자체가 안될 수 있음

- 단점2) RNN은 네트워크 계층별로 미니 정규화를 적용해야 하므로 모델 복잡해짐

▼ 8.3.2. 드롭아웃을 이용한 성능 최적화

코드 참고

▼ 8.3.3. 조기 종료를 이용한 성능 최적화

코드 참고